VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	**DEPARTMENT OF INFORMATION TECHNOLOGY** **TE Major Project-I Approval Form AY:2025-26**

Date:- 26-03-2026

Group ID:- IT65

1] Project Details:

1.1 Group Details:

Sr. No.	Roll No.	Name	Contact No	Sign
1	23101A0022	Aryant Jadhav	+91 9920925731	
2	23101A0023	Kashyap Nathoo	+91 8595511923	
3	23101C0056	Gauri Kamble	+91 7506150363	
4	23101C0048	Devashree Pavle	+91 9082064708	

1.2Name of Project Guide: Dr.Rugved Deolekar

1.3 Title of Ideas Presented:

Sr. no.	Topic Name	Domain	Status (Accepted /Rejected)
1	**Multi-Branch Deep Learning Trading System**	**Quantitative Finance and Artificial Intelligence**	
2	**DeepFake Detection and Analysis**	**Cyber Security**	
3	**License Plate Recognition System**	**Computer vision and image processing**	

2] Evaluation by Department Panel:

2.1 Idea1:

Sr. No.	**Parameters**	**Ratings (0-5)**
1	Quality of project idea, Innovativeness/Novelty	
2	Clarity of project idea	
3	Literature Survey	
4	Implementation Feasibility	
5	Communication skill & Presentation	

2.2 Idea2:

Sr. No.	**Parameters**	**Ratings (0-5)**
1	Quality of project idea, Innovativeness/Novelty	
2	Clarity of project idea	
3	Literature Survey	
4	Implementation Feasibility	
5	Communication skill & Presentation	

2.3 Idea3:

Sr. No.	**Parameters**	**Ratings (0-5)**
1	Quality of project idea, Innovativeness/Novelty	
2	Clarity of project idea	
3	Literature Survey	
4	Implementation Feasibility	
5	Communication skill & Presentation	

Remarks If any:

Signature of Guide:	Name and Signature of the Expert 1
Name and Signature of the Expert 2	Name and Signature of the Expert 3

3] Topic Details:

3.1 Abstract: Multi-Branch Deep Learning Trading System

This project presents a Multi-Branch Deep Learning Trading System designed to perform intelligent, data-driven financial market analysis and decision-making. The system integrates multiple specialized neural network models, each focusing on a distinct aspect of trading, including short-term execution, long-term forecasting, trend memory, inter-stock relationships, and market sentiment analysis.

By combining architectures such as CNN-LSTM with attention mechanisms, Temporal Fusion Transformers, Graph Neural Networks, and reinforcement learning-based sentiment models, the system is capable of processing diverse financial data sources simultaneously. It leverages advanced data engineering techniques like TFRecords optimization and Gramian Angular Field encoding to improve computational efficiency and model accuracy.

The multi-branch design enables the system to capture both micro-level patterns (like short-term price movements) and macro-level dynamics (such as sector correlations and market sentiment). Additionally, it incorporates uncertainty estimation and risk-aware decision-making to enhance trading reliability.

Overall, the system aims to provide a scalable, high-performance, and intelligent framework for quantitative trading by combining predictive analytics, relational modeling, and real-time data processing.

3.2 Literature Survey:-

The domain of stock market forecasting has evolved significantly from traditional statistical methods to advanced deep learning-based architectures. Financial markets are inherently volatile, non-linear, and influenced by multiple dynamic factors, making accurate prediction a complex problem. Early approaches such as AutoRegressive Integrated Moving Average (ARIMA), Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and Random Forests provided baseline solutions but failed to capture long-term dependencies and non-linear relationships in financial time-series data.With the advancement of deep learning, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) models became widely adopted due to their ability to model sequential data and handle the vanishing gradient problem. However, standalone LSTMs have limitations in capturing very long-range dependencies and often overlook localized patterns due to their global learning structure.

To overcome these challenges, recent research emphasizes hybrid and attention-based models. Architectures such as Hybrid CNN-LSTM with Attention combine spatial feature extraction and temporal learning, while attention mechanisms enable the model to focus on the most relevant time steps, improving prediction accuracy. Similarly, Temporal Fusion Transformers (TFT) have gained prominence for multi-horizon forecasting by integrating static and time-varying inputs and generating probabilistic predictions using quantile loss functions. Another advancement, Transductive LSTM (TLSTM), introduces adaptive learning by dynamically adjusting model parameters based on recent data patterns, making it more responsive to real-time market changes.

A significant limitation in earlier models was treating stocks as independent entities. Modern approaches address this using Graph Neural Networks (GNNs) such as Temporal Relational Networks (TRNet), which model inter-stock relationships using sector-wise correlations or statistical measures. This relational modeling improves prediction accuracy by incorporating market-wide dependencies.

In addition, sentiment analysis has become a critical component of forecasting systems. Tools like FinBERT analyze financial text data to extract market sentiment, which strongly influences short-term price movements. To address the challenge of imbalanced sentiment data, reinforcement learning techniques such as Off-Policy Proximal Policy Optimization (PPO) are used, assigning higher importance to rare but impactful negative events like market crashes.

Efficient data engineering and feature extraction techniques further enhance model performance. Methods such as Gramian Angular Field (GAF) convert time-

series data into image representations, enabling CNNs to capture visual patterns effectively. Feature representations like CULR provide better accuracy than traditional OHLC data. Additionally, optimized data pipelines using TFRecords, parallel processing, and mixed precision training significantly improve computational efficiency and scalability.

3.2 Objectives:

1. Architect a Low-Latency, Memory-Optimized Data Harvester
To develop a lightweight Python extraction script (utilizing a Zero-Pandas architecture) capable of maintaining a live WebSocket connection to the Sharekhan API. The system must reliably capture and buffer high-frequency tick data (OHLCV, Order Book, Open Interest) for 226 F&O stocks without exceeding the 2GB RAM limit of an AWS EC2 t3.small instance.

2. Implement Dynamic Contract Filtering Algorithms
To engineer intelligent filtering mechanisms, specifically a 5% Spot Anchor Bracket and a Smart Expiry Sorter. These algorithms aim to eliminate thousands of illiquid, out-of-the-money options contracts from the data feed, drastically reducing computational load and preventing API rate-limit violations.

3. Enable Real-Time Sector Mapping
To integrate an internal RAM-based mapping system that dynamically categorizes incoming stock ticks into 10 distinct market sectors (e.g., Private Banks, IT, Realty) and 3 core indices. This objective ensures the data is perfectly structured for downstream relational machine learning models without altering the raw CSV extraction speed.

4. Automate Cloud Infrastructure and Remote Management
To achieve a "Set-and-Forget" server lifecycle. This involves utilizing AWS EventBridge for automated market-hour booting, and developing a secure Telegram Commander Bot to inject daily 2FA API tokens remotely via a mobile device, eliminating the need for manual SSH logins.

5. Design a Cost-Effective Data Lake Archival System
To build a "Smart Gatekeeper" script that automatically compresses (.gz) and offloads batched CSV data to an Amazon S3 storage bucket every three verified trading days. This objective guarantees infinite data scaling for machine learning while keeping EC2 local storage and AWS billing strictly optimized.

6. Engineer Institutional-Grade Quantitative Features
To construct a post-market Greek Engine utilizing the py_vollib Black-Scholes solver. This engine will reverse-engineer Implied Volatility (IV) and calculate complex option Greeks (Delta, Gamma, Theta, Vega, Rho) to mathematically enrich the raw tick data.

7. Deploy Multimodal Deep Learning for Price Forecasting
To ultimately feed the archived, sector-mapped, and Greek-enriched TFRecord data into an advanced Time-Series AI framework. The objective is to utilize an ensemble of LSTMs, CNNs, and Transformers (like the Temporal Relational Network) to predict ultra-short-term (1-minute and 5-minute) market movements and volatility.

3.3 Project flow:

Phase 1: Pre-Market Boot Sequence & Authentication (8:45 AM)

1. **Server Wake-Up:** AWS EventBridge automatically powers on the Ubuntu EC2 (t3.small) instance at 8:45 AM IST.
2. **Service Initialization:** Linux systemd automatically launches the background Python services, including the Telegram Commander Bot and the Tick Harvester.
3. **Sector Mapping:** The harvester reads the sector_symbols.json file and maps all target stocks (e.g., Private Banks, IT, Auto) into the server's RAM for ultra-fast internal sector tagging.
4. **Remote Authentication:** You open the Telegram app on your phone and message your custom bot with the new daily Sharekhan access_token. The bot authenticates your Chat ID, updates the config.json file, and instantly restarts the harvester to apply the token.

Phase 2: Dynamic Contract Filtering (9:00 AM - 9:14 AM)

1. **Spot Price Anchoring:** Before subscribing to the live WebSocket, the system queries the NSE/BSE Cash market to fetch the exact, current underlying prices for all target symbols.
2. **Strike Bracket Calculation:** The system mathematically calculates a strict +/- 5% bracket around that spot price. Any options contract falling outside this bracket is completely discarded to save memory**.**
3. **Smart Expiry Selection:** The system chronologically sorts available expiry dates, automatically selecting the nearest weekly expiry for indices (Nifty/Sensex) and the current month's expiry for BankNifty.

Phase 3: Live Market Extraction (9:15 AM - 3:30 PM)

1. **WebSocket Streaming:** The system establishes a live connection to the Sharekhan API, pulling in continuous High-Frequency Trading (HFT) tick data

(OHLCV, Best Bid/Ask, Open Interest).

2. **Zero-Pandas Memory Buffer:** To prevent the 2GB server from crashing, ticks are rapidly appended to an internal Python list.
3. **Batch Writing:** The exact moment this list hits 500 ticks, the system flushes the data directly into a single daily CSV file (ticks_YYYY-MM-DD.csv) and instantly clears the RAM. This loop continues uninterrupted until the market closes.

Phase 4: Scheduled Data Archival (Post-Market)

1. **Smart Gatekeeper Verification:** At 9:00 PM, a Linux cron job triggers the s3_archiver.py script. The script uses the pandas_market_calendars library to verify if the day was an actual trading day (ignoring weekends and holidays).
2. **Batch Counting:** The script counts the local CSV files. If there are fewer than 3 files, it goes back to sleep.
3. **Compression & Upload:** Once 3 full trading days are collected, the script compresses the CSVs into .gz files and securely uploads them to your Amazon S3 Data Lake using the EC2 server's IAM Role.
4. **Self-Cleaning:** Upon a verified successful upload, the local files are deleted from the EC2 hard drive, resetting the system for the next cycle.

Phase 5: Local Transformation & Greek Engineering (Offline)

1. **Data Retrieval:** The archived .gz files are downloaded from the Amazon S3 bucket to your local Windows PC (utilizing your RTX 3050 GPU environment).
2. **Black-Scholes Processing:** The options_greeks_calculator.py script runs the raw data through the py_vollib C-extension.
3. **Feature Enrichment:** The script calculates Implied Volatility (IV), Theoretical Price, Time Value, and the core Option Greeks (Delta, Gamma, Theta, Vega, Rho), appending these institutional metrics to the dataset.

Phase 6: Machine Learning & Prediction (The Future Engine)

1. **TFRecord Conversion:** The enriched CSV data is converted into the highly optimized TFRecord format to maximize GPU training speed.
2. **Multimodal AI Training:** The data is fed into a multi-branch neural network architecture
    - **HCLA (Hybrid CNN-LSTM):** Analyses visual and sequential short-term order book patterns.
    - **TFT (Temporal Fusion Transformer):** Generates probabilistic price forecasts using Quantile Loss.
    - **TRNet (Temporal Relational Network):** Uses the sector mappings established in Phase 1 to predict price movements based on how correlated peer stocks (e.g., HDFC and ICICI) are moving.

3.4 Technology stack:

1. Core Programming Languages

- **Python 3.x:** The primary language powering the entire backend, from the live WebSocket harvester to the S3 archiver and machine learning models.
- **C / C++:** Operates under the hood to provide lightning-fast execution speeds for the py_vollib Black-Scholes solver and TensorFlow data processing.

2. Cloud Infrastructure & DevOps (AWS)

- **Amazon EC2 (Ubuntu Linux):** A t3.small (2GB RAM) instance acting as the live extraction server.
- **Amazon S3:** Used as the infinite cloud data lake for storing compressed high-frequency tick data.
- **AWS Event Bridge:** The automated serverless scheduler that boots the EC2 instance at 8:45 AM daily.
- **AWS IAM:** Handles secure, password less authentication between the EC2 server and the S3 bucket.
- **Linux native tools:** system (for running the harvester as a background service) and corn (for scheduling the S3 archiver script).

3. Brokerage & Communications APIs

- **Sharekhan API:** The primary data source, utilizing WebSockets for live, millisecond-level tick streaming and REST endpoints for authentication and order routing.
- **SMTP (smtplib):** Configured for the boot-sequence failsafe to instantly send email alerts if the script fails to resolve market symbols.

4. Data Engineering & Storage

- **Zero-Pandas Standard Library:** json and csv modules are strictly used on the EC2 server to prevent memory crashes during live extraction.
- **Boto3:** The official AWS SDK for Python, used by the archiver to transfer files to S3.
- **Gzip:** Used for high-ratio data compression (.gz) to minimize network bandwidth and cloud storage costs.
- **Pandas Market Calendars (pandas_market_calendars):** Provides institutional-grade awareness of NSE/BSE holidays without triggering anti-scraping firewalls.

5. Quantitative Finance & Mathematical Libraries

- **Py_vollib:** A highly optimized C-extension library used on the local machine to calculate Implied Volatility (IV) and Option Greeks (Delta, Gamma, Theta, Vega, Rho) via the Black-Scholes-Merton model.
- **Pandas:** Used strictly post-market in the local environment for heavy data transformation, grouping, and feature engineering.

6. Machine Learning & Deep Learning Frameworks

- **TensorFlow:** Specifically utilized for the TFRecord data format, which drastically optimizes memory consumption and data loading speeds during model training.

- **Deep Learning Libraries (TensorFlow/Keras & PyTorch):** Powering the multi-branch neural architectures, including the Hybrid CNN-LSTM with Attention (HCLA), the Temporal Fusion Transformer (TFT), and the Temporal Relational Network (TRNet).
- **Hugging Face (Transformers):** Utilized for deploying FinBERT, an LLM trained strictly on financial text for sentiment and risk analysis.
- **Reinforcement Learning:** Off-Policy PPO (Proximal Policy Optimization) algorithms for dynamic reward adjustments during imbalanced classification training.

3.5 References:

1. *Bhanujyothi, H. C., & Jacob, I. J. (2025). A Hybrid CNN-LSTM Attention-Based Deep Learning Model for Stock Price Prediction Using Technical Indicators. Engineering, Technology & Applied Science Research, 15(5): 28012-28017.*

2. *Patel, M., Jariwala, K., & Chattopadhyay, C. (n.d.). Advancing Indian Stock Market Prediction with TRNet: A Graph Neural Network Model.*

3. *Aarthi, K. C., & Katiravan, J. (n.d.). Comprehensive Review on Predictive Analytics in Indian Stock Market with Hybrid Deep Learning Model.*

4. *Gona, N. S. R., Aunugu, D. R., Methuku, V., Agrawal, M., & Myakala, P. K. (2025). Hybrid LSTM-Transformer Model for Stock Market Prediction: A Deep Learning Approach. IEEE.*

5. *Pavithra, K., Varshini, P., Anu, P., Mohanapriya, D., & Soundharya, S. (2024). Machine Learning Based Indian Stock Market Forecasting. First International Conference on Data, Computation and Communication.*

6. *Patel, M., Jariwala, K., & Chattopadhyay, C. (n.d.). SM2PNet: A Deep Learning Approach Towards Indian Stock Market Movement Prediction.*

7. *Ismail, S., Pattani, R. V., & Chacko, S. C. (2025). SMP-TFT: A Transformer-Based AI Framework for Stock Market Prediction. IEEE.*

8. *Peivandizadeh, A., Hatami, S., Nakhjavani, A., Khoshsima, L., Qazani, M. R. C., Haleem, M., & Alizadehsani, R. (2022). Stock market prediction with transductive long short-term memory and social media sentiment analysis. IEEE Access.*

3.6 Timeline of project action plan:

Phase	Month	Key Activities & Milestones
Phase 1: Data Engineering	Month 1	**Data Formatting & Encoding**: Transition all historical data into TFRecords to maximize GPU utilization. Encode time-series data into 2D images using GAF and use CULR feature selection to push CNN pattern recognition accuracy above 92%.
Phase 2: Tactical Branch	Month 2	**Short-Term Execution Engine**: Implement the HCLA (Hybrid CNN-LSTM with Attention) architecture. Integrate Attention-weighted Hybrid Feature Extraction (AHFE) and Dynamic Error Feedback Adjustment (DEFA) to actively refine live predictions.
Phase 3: Strategic Branch	Month 3	**Context & Forecasting Logic**: Develop the Temporal Fusion Transformer (TFT) with a Quantile Loss Function. Configure the model to produce Probabilistic Forecasts to explicitly quantify market uncertainty for risk management.
Phase 4: Memory Branch	Month 4	**Long-Term Trend Analysis**: Deploy a TLSTM (Transductive LSTM) or a Hybrid LSTM-Transformer. Program the logic to dynamically modulate parameters based on immediate temporal patterns to capture short-term volatility and long-term momentum.
Phase 5: Relational Branch	Month 5	**Sector Correlation Mapping**: Construct the SM2PNet / TRNet (Temporal Relational Network using GNNs). Build an adjacency matrix using Pearson's correlation coefficient to map linkages and predict movements

Phase	**Month**	**Key Activities & Milestones**
		based on highly correlated peers.
Phase 6: Sentiment & Risk	Month 6	**Imbalanced Data Processing**: Implement FinBERT combined with Off-Policy PPO to analyze financial text. Adjust the reward mechanism and utilize a Combo Loss (CL) function to amplify rare samples and manage imbalanced classification.
Phase 7: Testing & Refinement	Month 7-8	**System Integration & Backtesting**: Combine all neural network branches into the final, institutional-grade Multi-Branch Deep Learning Trading System. Conduct extensive system testing and finalize architecture upgrades.